

Online Intrusion Detection for Industrial Cyber-Physical Systems Based on Self-Supervised Predictive Model

Sardar Shan Ali Naqvi , Chunjie Zhou , Peihang Xu , *Student Member, IEEE*, Yahui Li, and Muhammad Uzair

Abstract—Industrial cyber-physical systems (CPS) demand a reliable intrusion detection despite evolving operational conditions and a lack of labeled attack data. This article proposes a novel context–target predictive self-supervised framework for online intrusion detection in industrial CPS. The models normal behavior by predicting future sensor states from past context, allowing detection of anomalies as large prediction errors. To address concept drift, we incorporate a trust-aware online adaptation mechanism and a robust, statistics-based thresholding strategy that adjusts in real time. We validate the method on a fluid catalytic cracking unit simulation, where it achieves high precision and recall under fully online, unlabeled conditions, outperforming baseline models. The proposed framework provides an adaptive and lightweight intrusion detection systems suitable for autonomous industrial CPS monitoring without human intervention or costly labeling.

Index Terms—Cyber-physical security, intrusion detection, online learning, self-supervised learning (SSL).

I. INTRODUCTION

INDUSTRIAL control systems (ICSs) managing critical infrastructure are increasingly vulnerable to cyber attacks [1]. The Stuxnet incident demonstrated how malware can infiltrate programmable logic controllers and physically damage processes [2]. In recent years, ICS-targeted attacks have increased [3], reflecting the growing exposure of operational technology to cyber threats [4]. Modern ICS integration with enterprise networks expands the cyber attack surface [5].

Traditional intrusion detection systems (IDSs) in industrial networks typically rely on known attack signatures or supervised models trained on historical labeled data [6]. However, obtaining comprehensive labeled ICS attack datasets remains a significant

challenge, because attacks are rare and identifying anomalies requires domain expertise [7]. Moreover, offline supervised IDS often struggles under concept drift [8]. Changes in sensor base-lines, control set-points, or production regimes can cause false alarms or missed detections if the model is not regularly updated. Yet, frequent retraining is impractical given the operational constraints of industrial environments [9].

Recent research has explored unsupervised and self-supervised approaches for ICS intrusion detection, eliminating the need for labeled attack data and enabling online adaptation [10]. These methods model normal behavior and flag deviations as anomalies. For example, autoencoder (AE)-based IDS attempts to reconstruct sensor measurements and raise an alarm when the reconstruction error is high [11]. Kitsune, an ensemble of AEs, incrementally learns from streaming data in real-time for internet of things (IoT)/ICS scenarios [12]. However, purely reconstructive approaches may fail to capture the temporal dependencies crucial for detecting context-based anomalies.

In self-supervised predictive learning, the model learns to predict future observations from past context. This approach leverages the time-series nature of cyber-physical systems (CPS) by predicting the expected system behavior, the model can identify anomalies as significant prediction errors. Hundman et al. [13] demonstrated the effectiveness of an LSTM-based predictor with dynamic nonparametric thresholding for spacecraft anomaly detection. For ICS, a predictive model can learn multivariate sensor dynamics and flag events where actual sensor readings deviate from the predicted normal behavior. Such prediction models, trained on the inherent structure of data, eliminate reliance on attack labels. However, predictive IDS faces a challenge while deciding when to trust new data for model updates. Indiscriminately learning from all incoming data risks adopting anomalous patterns as normal, especially under concept drift.

To address these challenges, we propose a self-supervised context–target predictive IDS that learns to predict future sensor readings from recent context. Anomalies are flagged when prediction errors exceed an adaptive threshold. The core idea is a two-module deep model: a *context encoder* that processes a window of recent sensor data (the context) and a *future predictor* that estimates the next readings (the target). By training on normal operation data to minimize prediction error, the model learns a compact representation. At run-time, the prediction error serves as an *anomaly score*. Larger errors indicate a possible deviation from learned normal patterns.

Received 7 January 2026; revised 28 February 2026; accepted 4 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62127808, Paper no. TII-25-9235. (Corresponding author: Sardar Shan Ali Naqvi.)

Sardar Shan Ali Naqvi, Chunjie Zhou, Peihang Xu, and Yahui Li are with the College of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: d_shanali@hust.edu.cn; cjiezhou@hust.edu.cn; xpei-hang@hust.edu.cn; yahui@hust.edu.cn).

Muhammad Uzair is with the School of Computer and Mathematical Sciences, University of Adelaide, Adelaide SA 5005, Australia (e-mail: muhammad.uzair@adelaide.edu.au).

Digital Object Identifier 10.1109/TII.2026.3691338

Building upon the prediction model, our framework introduces a *trust-aware online learning mechanism* inspired by the concept of model self-confidence in [14]. We define a *trust score* that measures the confidence of IDS in each detection decision based on the statistical distribution of recent anomaly scores. If the model produces a low anomaly score for a new data point, and if the score is within the range of past “normal” scores, then the decision can be trusted. Only in these high-trust situations do we use the model’s output to label data as benign and update the model. This strategy prevents the system from corrupting itself by learning from unverified or borderline data points. This acts as online calibration, allowing adaptation only when the model is confident. The contributions of this work are as follows.

- 1) We propose a novel self-supervised industrial IDS architecture that learns a predictive model of sensor behavior, rather than a reconstructive model, which improves detection of context-dependent anomalies.
- 2) We introduce a trust-based adaptation strategy with mathematical definitions for anomaly scoring, threshold selection, and trust computation, which achieves a balance between accuracy and efficiency, significantly reducing false positives while maintaining high recall.

The rest of this article is organized as follows. Section II reviews background and related work. Section III formalizes the problem and objectives of online intrusion detection. Section IV details the proposed context–target predictive framework, trust score computation, and online update rules. Section V describes the experimental setup, datasets, baseline methods, and evaluation metrics. Section VI presents results and a discussion. Finally, Section VII concludes this article and outlines future research directions.

II. BACKGROUND AND RELATED WORK

Recent advances in industrial CPS security include various approaches to detect cyber threats at the process level [15]. In this section, we highlight key developments that inform our approach, including anomaly detection techniques in ICS, self-supervised learning (SSL) methods, and online adaptation mechanisms.

A. Anomaly Detection in Industrial CPS

ICS anomaly detection has traditionally based on physical models or static thresholds [16], but the increasing complexity of sensor data and attack vectors has shifted the focus to data-driven approaches [17]. Supervised IDS models [18] achieve high accuracy on known attacks but require labeled data and frequent retraining, which is a practical limitation in ICS environments [19].

Unsupervised deep learning, particularly reconstruction-based AEs, is widely adopted for ICS anomaly detection [11], [20], [21]. These models learn a baseline of normal behavior by minimizing reconstruction loss, with large deviations indicating anomalies. Du et al. [22] enhanced AE robustness via synthetic anomaly injection, while long short-term memory (LSTM)-AE hybrids [23] incorporate sequence prediction. However, such methods often rely on fixed thresholds and struggle with contextual or subtle sequential anomalies, especially under concept drift.

Recent work also exposes weaknesses in ICS anomaly detection pipelines. Turrin et al. [24] showed that unstable datasets

and train/test distribution mismatches can cause false alarms and degraded IDS performance, motivating checks of distributional consistency (e.g., with kolmogorov–smirnov (KS) and kullback–leibler (KL) metrics) before training. Erba and Tippenhauer [25] further demonstrated that reconstruction-based and other model-free detectors are vulnerable to concealment attacks that induce slow, plausible drifts. These findings motivate predictive, temporally aware IDS with adaptive calibration to detect concealed deviations while avoiding contamination during anomalous periods.

B. Predictive SSL

Predictive SSL offers a more flexible alternative by predicting future sensor readings from past context. This leverages temporal dependencies and detects anomalies as prediction errors. Hundman et al. [13] used an LSTM predictor with dynamic thresholding for spacecraft telemetry, demonstrating robustness to noise and nonstationarity. Similar strategies have emerged in ICS [26], [27], where context–target SSL improves anomaly detection without requiring attack labels.

Compared to reconstruction-based models, predictive SSL inherently captures temporal consistency and can detect anomalies that maintain plausible values but disrupt normal sequence dynamics. However, these models still require careful threshold tuning and may degrade under concept drift without adaptive mechanisms.

C. Online Adaptation and Trust Mechanisms

Maintaining IDS performance over time requires online adaptation to evolving ICS conditions. Continuous learning (e.g., retraining on sliding windows [28], [29]) risks model poisoning if attack samples are mislabeled as normal. To mitigate this, trust-aware adaptation techniques have emerged. Nakip and Gelenbe [30], [31] proposed computing a model trust score based on statistical consistency, allowing learning only from high-confidence samples. This stabilizes performance and prevents concept drift corruption.

Our framework incorporates a similar notion of trust, tailored to the context of predictive anomaly detection in ICS. We define a trust score r_t for each new sample at time t as a function of the sample’s anomaly score relative to the expected distribution of normal scores. r_t is high if the prediction error is within normal bounds and low otherwise. Only when r_t exceeds a chosen threshold do we use that sample in further training of the model. This selective update rule ensures that during sustained attacks or major anomalies, when many r_t values would drop below the threshold, the model effectively pauses its adaptation, thus preserving the integrity of previously learned normal patterns. In Section IV-C, we mathematically define the trust score and the online update algorithm. We note that similar ideas of filtering updates based on anomaly likelihood have been applied in data stream mining [32], but our formulation integrates directly with the predictive error metric and adaptive threshold of the IDS.

Digital-twin IDSs may incorporate process-level mechanistic models, equipment or system-level models, and even 3-D representations of physical entities. Such multilayer twins can support detailed ICS security analysis when high-fidelity models are available [33]. The proposed IDS, however, operates purely at the process/field level and is fully data-driven, and it learns short-horizon dynamics in a latent space from normal data and

TABLE I
DESIGN CONSTRAINTS AND PERFORMANCE OBJECTIVES FOR THE PROPOSED ONLINE IDS

Constraint	Description
No labels	Operates without labeled data; decisions are made entirely through self-supervised learning.
Resource Efficiency	Must function under strict memory and processing limits typical of industrial edge devices.
Concept Drift	Capable of handling evolving system dynamics without manual intervention or retraining.
Objectives	Description
High Detection	Achieves strong precision, recall, F ₁ -score, and low FARs across diverse attack scenarios.
Self-adaptive	Continuously refines its internal model in response to data evolution, without external labels.
Stable	Ensures robust and stable performance through trust-regulated updates and noise resistance.

adapts online without requiring a physical or simulator-based model. Since multiple real ICS datasets are available, digital-twin-based data generation is unnecessary for this study. Our approach is therefore complementary to digital-twin IDSs, but does not rely on a complete digital twin for its operation.

III. PROBLEM FORMULATION

We consider a CPS with multiple sensors. The system generates a multivariate time-series $\{x_t\}_{t=1}^{\infty}$, where $x_t \in \mathbb{R}^d$ is the d -dimensional feature vector of sensor readings and actuator states at time t . Our goal is to continuously monitor these data and issue an alert whenever an anomalous behavior is observed, indicative of a cyber attack or malfunction, while adapting to normal operational changes over time.

Formally, at each time step t , the IDS must output a binary decision $y_t \in \{0, 1\}$, where $y_t = 0$ denotes *normal* operation and $y_t = 1$ denotes an *anomaly/attack*. This decision should be made based only on data up to time t and as promptly as possible. Table I presents the defined constraints and objectives for the online detection problem.

In the next section, we present our proposed framework that addresses these objectives by combining a predictive model for anomaly scoring with a trust-regulated online learning procedure.

IV. PROPOSED FRAMEWORK

Our intrusion detection framework consists of two main phases: an *Offline Pretraining Phase*, where the model learns an initial representation of normal behavior (this phase is optional if historical normal data is available, but beneficial for jump-starting the IDS) and an *Online Detection and Adaptation Phase*, where the model is deployed on streaming data and continuously updated in a self-supervised manner. Fig. 1 shows the overall architecture of the system.

A. Context–Target Prediction Model

At the core of our framework is an SSL task, given a window of recent context, predict the next readings of the system. L_c and L_t denote fixed lengths of the context window and prediction horizon, respectively, and do not depend on the time index t . The subscripts “ c ” and “ t ” indicate *context* and *target horizon*, whereas t in $\{x_t\}$ refers solely to the chronological time index. Thus, L_c and L_t simply specify how many past and future steps are used for prediction. In practice, L_c should span the

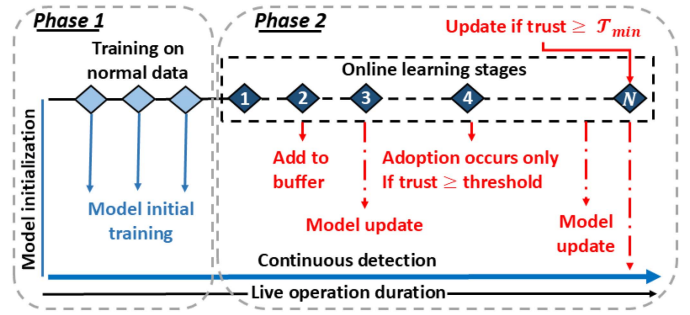


Fig. 1. Two-stages of proposed online IDS.

dominant short-term dynamics of the process. A useful guideline is to select L_c to cover approximately one to two characteristic settling times of the primary control loop, expressed in sampling steps (i.e., $L_c \approx T_s/\Delta t$), so that sufficient temporal context is provided without unnecessary computational overhead. The prediction horizon L_t determines the tradeoff between detection delay and noise robustness: $L_t = 1$ minimizes latency, whereas $L_t = 2-3$ can improve stability by capturing structured deviations across multiple steps. In our experiments, $L_c = 8$ and $L_t = 2$ provided a balanced operating point. We construct training samples from time-series data as follows:

$$X_t^{(c)} = [x_{t-L_c+1}, x_{t-L_c+2}, \dots, x_t] \in \mathbb{R}^{L_c \times d} \quad (1)$$

$$X_t^{(t)} = [x_{t+1}, x_{t+2}, \dots, x_{t+L_t}] \in \mathbb{R}^{L_t \times d} \quad (2)$$

where $X_t^{(c)}$ is the context (past L_c observations up to time t) and $X_t^{(t)}$ is the target (next L_t observations). The model comprises two neural networks: an encoder $f_\theta(\cdot)$ parameterized by θ and a predictor $g_\phi(\cdot)$ parameterized by ϕ . The encoder f_θ takes the multivariate context sequence $X_t^{(c)}$ and maps it to a latent representation $z_t = f_\theta(X_t^{(c)}) \in \mathbb{R}^m$, with m being the dimension of the latent space. This encoder can also be implemented as a 1-D convolutional network or an LSTM/gated recurrent unit (GRU) that compresses the window of length L_c into a fixed-length vector capturing the state. The predictor g_ϕ then processes z_t and predicts the latent representation of the future window

$$\hat{z}_{t+L_t} = g_\phi(z_t) \in \mathbb{R}^m. \quad (3)$$

To interpret this in the original feature space, we let the encoder also produce latent representations for the target window $X_t^{(t)}$. Specifically, we compute $z_t^{(\text{target})} = f_\theta(X_t^{(t)})$, i.e., feed the actual future data through the same encoder network. The training objective is to make the predicted latent $\hat{z}_{t+L_t}$ match the actual latent $z_t^{(\text{target})}$. We use a simple squared Euclidean distance in the latent space as the loss

$$\mathcal{L}_{\text{self}}(\theta, \phi) = \left\| g_\phi \left(f_\theta \left(X_t^{(c)} \right) \right) - f_\theta \left(X_t^{(t)} \right) \right\|_2^2. \quad (4)$$

where $z_t^{(\text{target})}$ is *not* an input to the predictor $g_\phi(\cdot)$ during inference. It is used only as the supervision signal for computing the self-supervised loss in (4). The arrow from $z_t^{(\text{target})}$ to the predictor in Fig. 2 therefore denotes the loss evaluation step, not an information flow into the predictor. This loss $\mathcal{L}_{\text{self}}$ is minimized over a collection of context–target the encoder and predictor jointly learn to predict the short-term future of the multivariate time series. The latent space $\mathbb{R}^m$ is an abstract

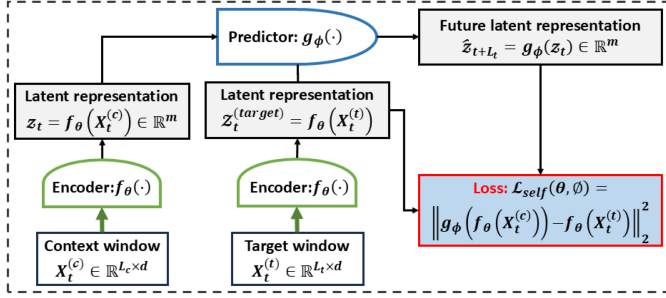


Fig. 2. Proposed context-target prediction model. The arrow from $z_t^{(\text{target})}$ to the predictor indicates the computation of the training loss and is not an input pathway to the predictor.

feature space where Euclidean distance corresponds to similarity of system states. Equation (4) encourages g_ϕ to correctly propagate the state z_t forward L_t steps. In practice, one could also have the predictor output directly the future raw features $\hat{X}_t^{(t)}$, but predicting in latent space has two benefits: 1) it reduces the output dimensionality (especially if $m \ll d \times L_t$) and 2) it allows the encoder to act as a dimensionality reducer/noise filter for the target as well. This setup is based on an AE stretched in time: f_θ compresses any sequence, and g_ϕ tries to generate the compression of the sequence shifted L_t steps ahead. The overall framework of proposed context-target prediction model is shown in Fig. 2.

Offline pretraining: In a controlled offline phase, we train $\{\theta, \phi\}$ to minimize $\mathcal{L}_{\text{self}}$ on a set of normal sequences. This can be done via stochastic gradient descent using mini-batches of $(X^{(c)}, X^{(t)})$ pairs. The outcome is an initial model that captures the nominal dynamics of the ICS. If no prior data are available, then this phase can be skipped; the model can start with random weights. In our experiments, we found that even pretraining on limited normal data significantly enhances initial accuracy.

B. Online Anomaly Detection

Once the predictive model is trained, it is deployed to monitor the live data stream. For each new time step t , we perform the following.

1) **Context and prediction:** We form the context window $X_t^{(c)}$ from the latest L_c points. The model computes $z_t = f_\theta(X_t^{(c)})$ and then $\hat{z}_{t+L_t} = g_\phi(z_t)$. This turns the predicted latent state for the horizon ending at $t + L_t$. When the actual data for time $t + 1$ through $t + L_t$ become available (which introduces a detection delay of L_t steps if we wait for the entire target window; in practice L_t is small, e.g., 1 or 2, so this delay is minimal), we compute $z_t^{(\text{target})} = f_\theta(X_t^{(t)})$.

The detection delay of the method equals the prediction horizon L_t and is therefore fully controllable. In our experiments, L_t was kept small (1 and 2), resulting in only a 1–2 s delay for fluid catalytic cracking (FCC) (1 Hz) and a 1–2 h delay for battle of the attack detection algorithms (BATADAL), which is acceptable because attacks evolve over several hours. If stricter responsiveness is required, then setting $L_t = 1$ reduces the delay to a single sampling step while preserving strong detection performance.

The predictor forecasts the latent representation of the *entire* future window $X_t^{(t)}$, allowing the model to capture short-term

dynamics and detect anomalies that may begin anywhere within $[t+1, t+L_t]$. Choosing $L_t = 1$ minimizes delay, while $L_t = 2$ offers improved noise robustness. Detected anomalies are attributed to time step t , consistent with standard predictive-model IDS practice; if finer localization is needed, then operators may set $L_t = 1$ or inspect the pointwise prediction errors.

Cyber-physical attacks often cause propagation effects, where a disturbance at time t pushes the process into an abnormal regime for several subsequent steps. Our method naturally captures this behavior, the anomaly score compares the predicted normal evolution with the observed trajectory, so persistent physical deviations remain detectable. Meanwhile, the trust mechanism prevents attack-corrupted samples from entering the update buffer, enabling the model to resume safe adaptation once the system returns to normal.

2) **Anomaly score computation:** We define the anomaly score at time t as the prediction error

$$a_t = \|\hat{z}_{t+L_t} - z_t^{(\text{target})}\|_2^2. \quad (5)$$

A higher a_t indicates that the model's prediction diverged from true values, suggesting an unexpected event. Note that $a_t \geq 0$, and $a_t = 0$ if and only if the predicted future exactly matches the predicted future in the latent space.

3) **Thresholding:** To decide if a_t signifies an anomaly, we use an adaptive threshold τ_t . Fixed thresholds are suboptimal because the scale of prediction error can vary with operating rules and slowly drift. We maintain a rolling window of the last W anomaly scores on recent legitimate data. Let $S_t = \{a_{t-W+1}, \dots, a_t\}$ denote this window of recent scores. We set

$$\tau_t = \text{Median}(S_t) + \lambda \cdot \text{IQR}(S_t) \quad (6)$$

where IQR is the interquartile range (75th percentile minus 25th percentile), and λ is a tunable multiplier (e.g., $\lambda = 1.5$) to adjust sensitivity. This threshold estimator is robust to outliers and adapts to the distribution of the anomaly score. We also apply an exponential smoothing to the anomaly scores a_t themselves to damp high-frequency fluctuations

$$\tilde{a}_t = \alpha \cdot a_t + (1 - \alpha) \cdot \tilde{a}_{t-1} \quad (7)$$

with a smoothing factor $0 < \alpha < 1$ (e.g., $\alpha = 0.1$). The smoothed score $\tilde{a}_t$ is used for threshold comparison and for computing S_t . Smoothing helps prevent noise from triggering false alarms.

Why median-IQR thresholding? Median and IQR are robust to transient spikes and skewed error distributions, making the threshold $\tau_t = \text{Median}(S_t) + \lambda \text{IQR}(S_t)$ stable even under mild nonstationarity or when a few anomalous values enter the window. This prevents threshold inflation caused by outliers and supports smooth adaptation during distribution changes.

4) **Decision:** The IDS outputs $y_t = 1$ (anomaly) if $\tilde{a}_t > \tau_t$, and $y_t = 0$ (normal) otherwise. An alarm is raised immediately when $\tilde{a}_t$ exceeds τ_t . The detection delay introduced by using a target window of size L_t means the alarm at time t truly pertains to an event that started L_t steps earlier, but for small L_t this is a negligible delay.

This anomaly detection process runs continually as new data arrive. By using the rolling window S_t for threshold calculation, the threshold self-adjusts: if the system enters a slightly different normal operating mode, the median and IQR of a_t will shift, raising τ_t to reduce false positives. While the model gets better through learning or the system stabilizes, the threshold tends to be lower to maintain sensitivity. The overall framework of the proposed online anomaly detection is shown in Fig. 3.

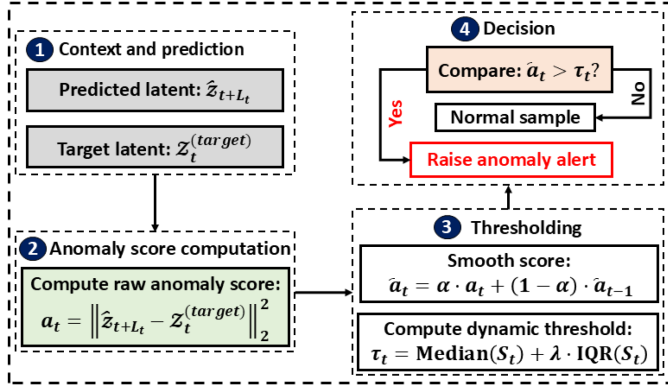


Fig. 3. Online Anomaly Detection.

C. Trust Score and Online Model Update

Detection alone is not sufficient. The challenge is to update the model (f_θ and g_ϕ) with new data in an unsupervised manner. We introduce a *trust score* $r_t \in [0, 1]$ to quantify confidence that x_t (and its context) represent normal behavior. The trust score is derived from the anomaly score in a statistically principled way

$$r_t = \max\left\{0, 1 - \frac{\tilde{a}_t}{\tau_t}\right\}. \quad (8)$$

The linear form $r_t = 1 - \tilde{a}_t/\tau_t$ provides a simple confidence measure: samples well below the threshold receive high trust, while samples near or above it receive low or zero trust. This smooth and lightweight mapping is suitable for real-time ICS and effectively gates online updates.

In this formulation, if the anomaly score $\tilde{a}_t$ is exactly at the threshold τ_t , then $r_t = 0$ (lowest trust, on the borderline of abnormal). If $\tilde{a}_t$ is well below the threshold, then r_t approaches 1. For example, if $\tilde{a}_t = 0$ (no error), then $r_t = 1$; if $\tilde{a}_t$ is half of τ_t , then $r_t = 0.5$. We also cap the minimum at 0 (if $\tilde{a}_t > \tau_t$, we treat it as 0 trust). This linear trust function is simple and captures the intuition: the more comfortably normal the sample looks (relative to the current threshold), the more we trust it as a true representation of the normal state.

We maintain an adaptation buffer $\mathcal{B}$ that will store recent trusted data samples. Specifically, at each time step t , if r_t exceeds a chosen high-confidence threshold $r_{\min}$ (for instance, $r_{\min} = 0.8$), we add the pair $(X_t^{(c)}, X_t^{(t)})$ to $\mathcal{B}$ as a new training sample with the self-supervised target. If $r_t < r_{\min}$, then we discard the sample to avoid model training with possibly anomalous data. The buffer has a fixed capacity or time range (accumulate up to B samples or cover the last T_B time steps). Once the buffer is full, or at periodic intervals, we trigger an online update of the model.

- 1) We use the accumulated buffer $\mathcal{B}$ to perform a mini-batch gradient descent step (or a few steps) on the loss (4), thereby fine-tuning f_θ and g_ϕ on the latest data. In our implementation, we treated this similarly to one epoch of training on the buffer data.
- 2) After updating, we clear $\mathcal{B}$ and continue accumulating next.

Clearing $\mathcal{B}$ only resets the temporary adaptation buffer used to form the next online update, the learned normal patterns are retained in the model parameters (θ, ϕ) , which are modified only by the performed gradient step. The parameter $r_{\min}$ controls the tradeoff between adaptation and caution. A higher $r_{\min}$ (closer to

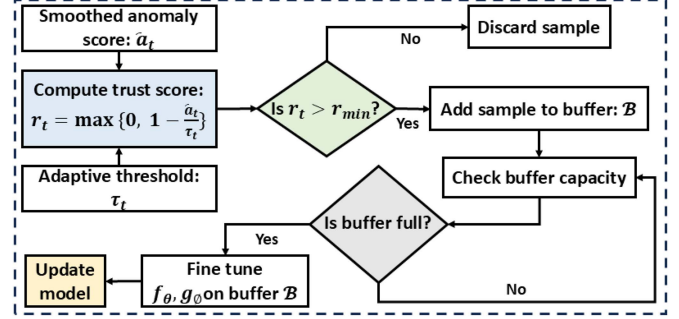


Fig. 4. Trust score and online model update process.

1) means only very safe, low-error samples are used for learning, which reduces the risk of using anomalous data but might slow adaptation. A lower $r_{\min}$ allows more samples to be used, enabling faster learning but with increased risk. In our experiments, we chose $r_{\min}$ empirically to balance these factors. To prevent the trust gate from freezing adaptation under benign concept drift, we track a freeze counter n_{freeze} that increments whenever no sample is admitted to $\mathcal{B}$, i.e., $r_t < r_{\min}$. If $n_{\text{freeze}} > T_{\text{freeze}}$, then we temporarily relax $r_{\min}$ within a conservative lower bound $r_{\min}^{\text{low}}$ to allow safe updates to resume. Once $\mathcal{B}$ begins filling normally again, $r_{\min}$ is restored to its nominal value. In practice, we did not encounter prolonged freezing in our case studies.

Another detail is adjusting the learning rate for online updates. Since the model should not deviate suddenly from prior weights with each small batch, we use a smaller learning rate for online fine-tuning than we used in offline training. We also employ a patience schedule (if consecutive updates produce little reduction in $\mathcal{L}_{\text{self}}$ on $\mathcal{B}$, we reduce the learning rate further). This prevents oscillations and helps converge to the new normal pattern smoothly. The trust score and online model update process are shown in Fig. 4.

The framework remains stable across broad hyperparameter ranges. L_c controls how much history is encoded; L_t balances delay and noise robustness; W sets threshold smoothness; λ adjusts sensitivity; α governs how quickly trust reacts to prediction errors. In practice, L_t and W have the most noticeable effects on operational behavior.

On FCC and BATADAL, varying $r_{\min} \in [0.7, 0.9]$, $\alpha \in [0.05, 0.2]$, and $\lambda \in [1.4, 1.6]$ changed F1-scores by less than 0.02, indicating robustness to moderate hyperparameter variation.

When $X_t^{(t)}$ contains intruded values, prediction error increases, and the trust score r_t drops below $r_{\min}$, preventing such samples from updating the model. The resulting mismatch between $\hat{z}_{t+L_t}$ and $z_t^{(target)}$ signals an anomaly, and normal adaptation resumes automatically once high-trust samples reappear.

Sliding windows naturally create overlapping context–target pairs, each aligned to a different prediction target. This overlap provides fine-grained temporal learning signals and does not harm adaptation: mini-batch updates and the fixed buffer size ensure that recent high-trust samples dominate the learning process.

D. Algorithmic Summary

Fig. 5 shows a simplified block diagram of the end-to-end workflow, including context encoding, prediction, anomaly scoring, trust evaluation, and online updates. Algorithm 1 provides a

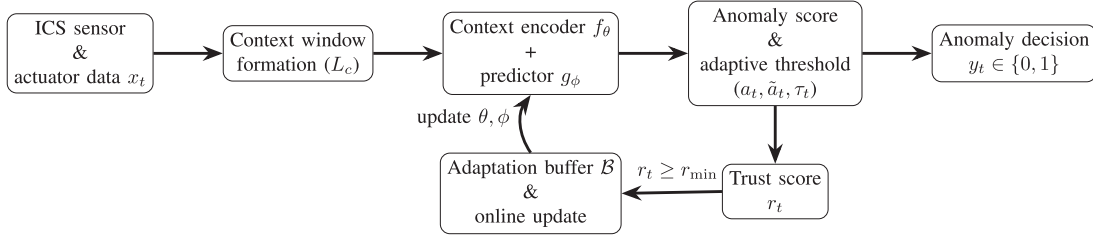


Fig. 5. Block diagram of the proposed context-target predictive IDS, showing context windowing, latent encoding, prediction, anomaly scoring with adaptive threshold, and trust-based online updates.

complete pseudocode of the online detection and adaptation process. At each time step, anomaly scores and trust are computed, decisions are made, and the model update is invoked as needed. The complexity per time step is $O(d \times L_c \times m)$ for encoding and $O(m^2)$ for prediction (dense layers in g_ϕ), which is typically quite fast (in our implementation, m was on the order of a few tens, L_c up to 10, and d of order 10^2). The adaptation update, which is more expensive, occurs infrequently (buffer-based) and can be done asynchronously if needed.

The above algorithm ensures that the IDS alternates between detection and learning. Importantly, the detection uses the latest model parameters at all times; as soon as the model is updated in the buffer, subsequent anomaly scores $\tilde{a}_t$ will reflect the updated model's expectations.

Algorithm 1 becomes operational only after a full context window has been observed. For the initial $t < L_c$ steps, the IDS runs in a warm-up mode: incoming samples are buffered, and no anomaly scores, thresholds, or trust values are computed. Once $t \geq L_c$, both $X_t^{(c)}$ and (after L_t additional samples) $X_t^{(t)}$ are well-defined, allowing Steps 2–21 of Algorithm 1 to execute normally. This warm-up phase corresponds to the common assumption that monitoring begins near normal operation; however, a strict “no intrusion” guarantee is not required. If an attack occurs during the warm-up, then elevated prediction errors will appear as soon as full windows become available, and the anomaly will be flagged starting at $t = L_c$. Thus, the only limitation is that detection cannot be defined before a complete context window exists, which is inherent to all window-based predictive IDS designs.

V. EXPERIMENTAL SETUP

We evaluate our proposed intrusion detection framework on two representative industrial datasets and compare it against baseline methods. The experiments are designed to answer:

- 1) How effectively does the context-target predictive model detect intrusions of cyber-physical attacks?
- 2) Does the trust-aware online adaptation improve performance over time, and how does it impact false positives/negatives?
- 3) How does our approach compare to other self-learning IDS and static baseline models?

A. Datasets

FCC unit dataset: As explained in [34], and used for adversarial features generation for VAE-based IDS [35], this dataset is generated from a simulated FCC unit, which is a critical

Algorithm 1: Online Predictive IDS with Trust-Based Adaptation.

Input: Pretrained encoder f_θ , predictor g_ϕ , trust threshold $r_{\min}$, buffer size B , smoothing factor α

Output: Real-time anomaly labels y_t and model updates

- 1 Initialize $\tilde{a}_0 \leftarrow 0$, buffer $\mathcal{B} \leftarrow \emptyset$;
 - 2 Initialize freeze counter $n_{\text{freeze}} \leftarrow 0$;
 - 3 **for each time step** $t = 1, 2, 3, \dots$ **do**
 - 4 Form context window $X_t^{(c)}$ from past L_c points;
 - 5 Encode: $z_t \leftarrow f_\theta(X_t^{(c)})$;
 - 6 Predict: $\hat{z}_{t+L_t} \leftarrow g_\phi(z_t)$;
 - 7 Upon receiving $X_t^{(t)}$ (future L_t points):
 - 8 $z_t^{(\text{target})} \leftarrow f_\theta(X_t^{(t)})$;
 - 9 Compute anomaly score:
 $a_t \leftarrow \|\hat{z}_{t+L_t} - z_t^{(\text{target})}\|_2^2$;
 - 10 Update smoothed score: $\tilde{a}_t \leftarrow \alpha a_t + (1 - \alpha)\tilde{a}_{t-1}$;
 - 11 Compute threshold τ_t from recent scores (Eq. 6);
 - 12 **if** $\tilde{a}_t > \tau_t$ **then**
 - 13 $y_t \leftarrow 1$; // Anomaly detected
 - 14 **else**
 - 15 $y_t \leftarrow 0$; // Normal
 - 16 Compute trust: $r_t \leftarrow \max(0, 1 - \tilde{a}_t/\tau_t)$;
 - 17 **if** $r_t \geq r_{\min}$ **then**
 - 18 Add $(X_t^{(c)}, X_t^{(t)})$ to buffer $\mathcal{B}$;
 - 19 $n_{\text{freeze}} \leftarrow 0$; // reset freeze counter
 - 20 **else**
 - 21 $n_{\text{freeze}} \leftarrow n_{\text{freeze}} + 1$; // no trusted sample
 - 22 **if** $n_{\text{freeze}} > T_{\text{freeze}}$ **then**
 - 23 $r_{\min} \leftarrow \max(r_{\min} - \Delta, r_{\min}^{\text{low}})$;
 - 24 // controlled relaxation
 - 25 **if** $|\mathcal{B}| \geq B$ **then**
 - 26 Train f_θ, g_ϕ for one epoch to minimize:
 $\mathcal{L}_{\text{self}} = \|g_\phi(f_\theta(X^{(c)})) - f_\theta(X^{(t)})\|_2^2$;
 - 27 Reset adaptation buffer (mini-batch cache):
 $\mathcal{B} \leftarrow \emptyset$;
-

component in oil refineries for converting heavy hydrocarbons into lighter fuels. The simulation mimics a realistic industrial control scenario with multiple sensor readings (temperatures, pressures, flow rates, etc.) and control signals. Key details are as follows.

- 1) *Number of sensors/features*: In total, 84 (including temperatures, pressures, flow meter readings, valve positions, etc.).
- 2) *Sampling rate and length*: The data are recorded at 1 Hz. We collected 50000 samples (approximately 13.9 h) of normal operation for training. For testing streaming performance, we simulate a real-time feed of 5000 samples (1.39 h) that includes a mix of normal periods and attacks.
- 3) *Attack scenarios*: We injected three types of intrusions during the test phase: (a) *Sensor drift*, in which one sensor's reading is gradually increased by a malicious bias over time; (b) *sudden fault*, in which a critical sensor is pushed at an extreme value for a short duration; and (c) *false data injection*, in which random noise or patterned false signals are added to several sensor readings to mimic a coordinated deceptive attack. These attacks are designed to challenge the IDS with both slow and abrupt deviations.

Although the FCC dataset does not include packet-level attacks (e.g., replay, man-in-the-middle (MITM), denial of service (DoS), and scans), such cyber actions ultimately exhibit as abnormal physical trajectories once malicious commands reach the programmable logic controller (PLC)/supervisory control and data acquisition (SCADA) layer. The injected drift, sudden faults, and false-data attacks therefore serve as process-level proxies for these higher level events and are appropriate for evaluating a predictive IDS focused on physical dynamics.

Among the five datasets used, FCC and the original BATADAL provide only process-level telemetry, whereas CICIOT2023, WDT (2021), and BATADAL-2 also include network-traffic traces. For consistency and because our attack scenarios target physical variables, we restrict our evaluation to the process-level, continuous-valued time-series signals across all datasets. This choice aligns with the design of our predictive IDS, which models short-horizon physical dynamics in latent space rather than packet-level fields. Protocol-layer features, although present in some datasets, were not used here but can be incorporated in future extensions toward joint network-process monitoring.

Each data sample is timestamped, and all features are continuous-valued. We normalized each feature to the range $[0,1]$ using min-max scaling based on the training set statistics, so that errors on different sensors are comparable in magnitude.

BATADAL Water Distribution Dataset: We also test our framework on the publicly available BATADAL dataset [36]. It consists of simulated data from a medium-sized water distribution network (C-Town) with multiple sensors (flows, levels, and pressures) over weeks, including both normal operation and cyber-attack scenarios. The BATADAL competition provided three datasets, a training set with only normal data, and two test sets with multiple staged attacks (insiders opening valves, closing pumps, etc.). We focus on the first test dataset, which contains seven distinct attack events over 14 days of hourly data. Key details are as follows.

- 1) *Number of Sensors*: In total, 43 sensors (combination of tank levels, pump statuses, and flow rates).
- 2) *Sampling Interval*: In total, 1 h per data point (thus 336 data points in two weeks).

- 3) *Attack Types*: The attacks include valve outages causing abnormal flow patterns, pump failures leading to pressure drops, and covert attacks replaying older sensor data to mask the true state. Each attack lasts several hours.

We use the provided normal training portion of BATADAL to pretrain our model, and then simulate online detection on the test set. For baseline comparisons on BATADAL, we note the results reported by participants in the original competition [36] and recent research [37], which show detection accuracies around 87%–91% for the best models. Our evaluation on this dataset is meant to assess the generalization of our approach beyond the FCC simulation and demonstrate adaptation to different ICS domains.

B. Baseline Methods

We compare our approach against the following baselines.

- 1) *Static AE*: A deep AE trained offline on the normal data of each dataset. It uses reconstruction error with a fixed threshold for anomaly detection. This represents a nonadaptive unsupervised baseline.
- 2) *Static LSTM predictor*: An LSTM sequence prediction model without online updates. It is trained offline on normal data and then used to predict one-step-ahead, with anomalies detected by prediction error over a static threshold (set via a validation subset). This baseline tests if our added components (adaptive threshold and trust-based updates) performed better than a simpler predictor.
- 3) *Incremental learner (no trust)*: An ablation of our method where the context-target model is updated online with every new sample or every small batch without using the trust filter. This is essentially continuous learning, where we assume all data are benign. Comparing this to our full method quantifies the impact of the trust mechanism.
- 4) *Supervised classifier*: Although not directly comparable (since it requires labels), we include a reference baseline where we train a random forest (RF) classifier on a portion of the attacks (for BATADAL, assume first few attacks labeled) to see the upper bound performance of a supervised approach on known attacks.

All methods use the same normalized inputs. The AE and LSTM architectures were tuned to have comparable capacity to our model (approximately similar number of parameters) for fairness. The static methods obviously cannot handle concept drift (except by raising more false alarms or missing new patterns), but they indicate initial detection capability.

C. Implementation Details

The context encoder f_θ in our model is implemented as a two-layer GRU network with 64 hidden units, followed by a dense layer to produce a latent vector z_t of dimension $m = 16$. The predictor g_ϕ is a two-layer feedforward network that takes the 16-dim z_t and outputs a 16-dim $\hat{z}_{t+L_t}$. We set context length $L_c = 8$ and target length $L_t = 2$ for both datasets. These values were selected based on short-term auto correlation analysis of sensor signals and the dominant settling time of the processes, ensuring that the context window captures the primary system dynamics while keeping detection delay minimal. The models were trained using the Adam optimizer. Offline pretraining used 75 epochs on the FCC normal data and 40 epochs on the BATADAL training set, with early stopping based on reconstruction loss.

TABLE II
PERFORMANCE COMPARISON OF IDS ON FCC AND BATADAL DATASETS

Dataset	Method	Acc.	Prec.	Rec.	F1	AUC	FAR	Delay	Time (s/sample)	Type	Labels?
FCC	Static AE	0.872	0.60	0.92	0.75	0.80	0.185	–	–	Offline	✓
	Static LSTM	0.885	0.70	0.92	0.80	0.88	0.120	–	–	Offline	✓
	Online (no trust)	0.823	0.75	0.70	0.72	0.89	0.235	4	0.0041	Online	✓
	Incremental (no trust)	0.837	0.78	0.74	0.76	0.89	0.205	3.5	0.0039	Online	✓
	RF	0.870	0.74	0.82	0.78	0.88	0.170	–	–	Offline	✓
	Proposed	0.903	0.94	0.85	0.90	0.953	0.056	2.1	0.0033	Online	✗
BATADAL	RF	0.885	0.80	0.88	0.85	0.90	0.130	–	–	Offline	✓
	Challenge Winner [36]	0.862	0.79	0.77	0.78	0.84	0.100	–	–	Offline	✓
	Online (no trust)	0.864	0.84	0.78	0.81	0.91	0.095	2	0.0039	Online	✓
	Incremental (no trust)	0.872	0.87	0.76	0.81	0.91	0.085	1.8	0.0037	Online	✓
	Proposed	0.889	0.985	0.82	0.897	0.946	0.022	1 interval	0.0031	Online	✗

TABLE III
PERFORMANCE COMPARISON OF IDS ON CICIOT2023, WDT (2021), AND BATADAL-2 DATASETS

Dataset	Method	Acc.	Prec.	Rec.	F1	AUC	FAR	Delay	Time (s/sample)	Type	Labels?
CICIOT2023	Static AE	0.881	0.82	0.89	0.85	0.90	0.155	–	–	Offline	✓
	RF	0.890	0.85	0.86	0.85	0.91	0.123	–	–	Offline	✓
	Online (no trust)	0.835	0.77	0.75	0.76	0.87	0.210	4.2	0.0042	Online	✓
	Incremental (no trust)	0.852	0.80	0.78	0.79	0.88	0.180	3.8	0.0040	Online	✓
	Proposed	0.925	0.95	0.88	0.91	0.965	0.045	2.5	0.0030	Online	✗
WDT (2021)	Static AE	0.875	0.78	0.90	0.83	0.89	0.140	–	–	Offline	✓
	RF	0.882	0.82	0.87	0.84	0.90	0.125	–	–	Offline	✓
	Online (no trust)	0.860	0.83	0.76	0.79	0.90	0.105	2.3	0.0041	Online	✓
	Incremental (no trust)	0.871	0.86	0.77	0.81	0.91	0.090	2.0	0.0038	Online	✓
	Proposed	0.895	0.98	0.83	0.90	0.951	0.020	1.5	0.0032	Online	✗
BATADAL-2	RF	0.887	0.81	0.88	0.84	0.91	0.120	–	–	Offline	✓
	Online (no trust)	0.868	0.85	0.78	0.81	0.91	0.090	2.2	0.0038	Online	✓
	Incremental (no trust)	0.875	0.88	0.77	0.82	0.92	0.080	1.7	0.0036	Online	✓
	Proposed	0.892	0.981	0.83	0.899	0.948	0.021	1.2 interval	0.0031	Online	✗

Note: LSTM baseline omitted for WDT (2021) and BATADAL-2 due to long training time.

For online updates, we set the adaptation buffer size $B = 256$ samples for FCC (roughly 4–5 min of data at 1 Hz) and $B = 20$ for BATADAL (20 h of data at 1/h, since anomalies there are much more sparse). The trust threshold $r_{\min}$ was set to 0.8. The smoothing factor was $\alpha = 0.1$ and the window W for threshold median/IQR was 100 recent scores for FCC, and 24 for BATADAL (since BATADAL data are hourly, a one-day window). The anomaly threshold multiplier was $\lambda = 1.5$ by default. In a sensitivity analysis, λ values between 1.4 and 1.6 did not significantly change results; too low λ caused more false positives, too high missed some small anomalies.

All experiments were run on a machine with an Intel Xeon CPU at 2.30 GHz. Training offline took a few minutes per model. Online detection ran in real-time easily: the average inference time per sample was around 0.0033 s, which is well below 1-s cycle time in our highest rate data, indicating feasibility for deployment on an industrial controller or edge device with similar specs.

VI. RESULTS AND DISCUSSION

We first present the overall detection performance of our proposed IDS compared to baselines on the two datasets. We then analyze the effect of the trust mechanism and show how the system adapts over time. Finally, we discuss the computational overhead and practical implications. Following Turrin et al. [24], we first verified the statistical consistency of training and test data. For every sensor, KS tests and KL divergences showed no significant distribution shift across all datasets ($KS\ p > 0.05$ and $KL < 0.10$), and no postattack instability was observed. Thus, all features were retained for model training and evaluation. This validation step ensures that the IDS is not affected by hidden dataset instability or shift-induced vulnerabilities [24], [25].

From a computational standpoint, encoding and prediction operate in a 16-D latent space with short windows ($L_c = 8$ and $L_t = 2$). As reported in Tables II and III, the resulting per-sample inference time for the proposed IDS is approximately 0.003–0.004 s across all datasets, which is far below the 1 Hz sampling rate of FCC and negligible compared to the hourly sampling of BATADAL and BATADAL-2. Even when including trust computation and threshold updates, the IDS therefore runs comfortably in real time on typical industrial edge hardware without interfering with PLC/SCADA cycle times.

To assess how well the model captures nominal dynamics, we first evaluate prediction precision before any anomaly scoring. Across all datasets, the latent predictor attains normalized mean absolute error (NMAE) < 0.045 and normalized root mean square error (NRMSE) < 0.062 (normalized by feature dynamic ranges), indicating accurate short-horizon forecasting of normal behavior. Normal operational variations, such as setpoint changes or benign disturbances, produce only brief and moderate increases in prediction error. These remain within the adaptive median–IQR threshold, which smooths transient fluctuations. In contrast, cyber-induced anomalies generate sustained, structured deviations that consistently exceed the threshold. This separation allows the IDS to distinguish benign disturbances from attack-induced deviations while maintaining low false-alarm rates (FARs).

A. Detection Performance

Our self-supervised predictive IDS achieves high precision and recall across both datasets. On FCC, it reaches an F1-score of 0.90 with an average delay of about 2.1 s, which is near the theoretical minimum of $L_t = 2$ steps (see Section V). For

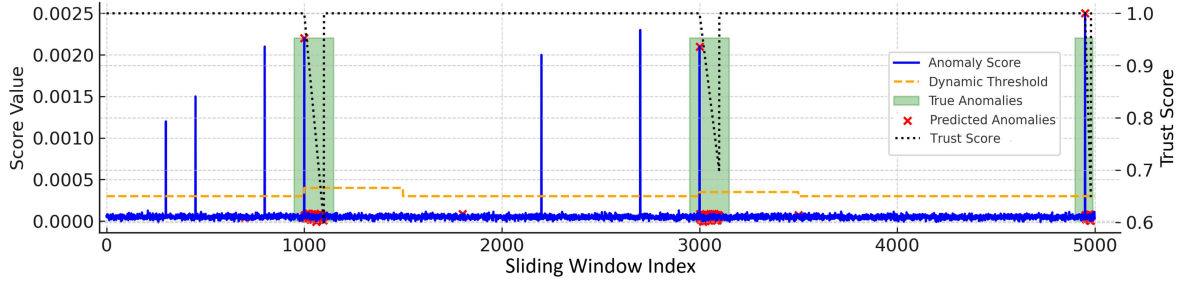


Fig. 6. Proposed IDS performance: Anomaly scores ($\tilde{a}_t$), dynamic threshold (a_t), predicted anomalies (red crosses), and trust score (r_t) over time. True anomaly regions are shaded green.

BATADAL, precision reaches 98.5% (FAR = 2.2%), critical for minimizing false alarms in water networks. The AUC 0.95 confirms strong separability between normal and attack states via latent-space reconstruction errors.

Table II shows the detailed performance comparison with offline and nonadaptive baselines. On FCC, precision improves by 34% over the static AE (0.60 versus 0.94) with a 76% FAR reduction compared to trustless online learning (0.235 versus 0.056). On BATADAL, our precision (98.5%) and FAR (2.2%) outperform the challenge winner [36] (F1: 0.78 versus 0.90). Fig. 6 shows the interplay of anomaly scores, adaptive thresholds, and trust-based updates.

To further evaluate the proposed IDS, we performed experiments on recent ICS public datasets, including CI-CIoT2023 [38], WDT (2021) [39], and BATADAL-2 [40]. We used the same model architecture, training procedure, and online adaptation strategy described in Section V, without dataset-specific tuning other than basic normalization. Performance comparison is shown in Table III.

Fixed thresholds of Static models (AE and LSTM) caused false alarms during distribution shifts. For static AE, F1 = 0.75, and Prec. = 0.60 shows its oversensitivity to benign fluctuations. While the LSTM baseline improved slightly (F1 = 0.80), its lack of adaptation led to missed detections during late-test sensor drift. Incremental learning with no trust, updates blindly during attacks, delaying detection to 4 s. While our trust mechanism stopped updating during anomalies, preserving model integrity (see Fig. 6).

Fig. 7 presents two detection scenarios on the FCC dataset. Fig. 7(a) shows a sudden fault attack on a pressure sensor for 18 s. Adaptive threshold τ_t reached 0.03 and the anomaly score $\tilde{a}_t$ spikes sharply between 7.4 to 14 s during a sudden fault attack on a pressure sensor. The model produced a high anomaly score for a few seconds after the fault ended, as the sudden normal was unexpected, and our predictor had not yet adjusted. The trust prevented training during the fault.

Fig. 7(b) shows a slow drift attack, where a temperature sensor is gradually manipulated. As the sensor readings deviate from the IDS predictions, the anomaly score rises. Initially, updates continue since the score remains below the adaptive threshold τ_t , which increases slowly up to 0.05. When the anomaly score nears τ_t , the trust score r_t drops below 0.8, preventing IDS from updating. This prevents the further rise in τ_t , despite ongoing drift. Eventually, the anomaly score exceeds τ_t , triggering detection, by which time the sensor had drifted 10% from baseline, although detection is slightly delayed, early alerts risk false positives. The trust mechanism ensures that once uncertainty is detected, drift is not learned as normal. The

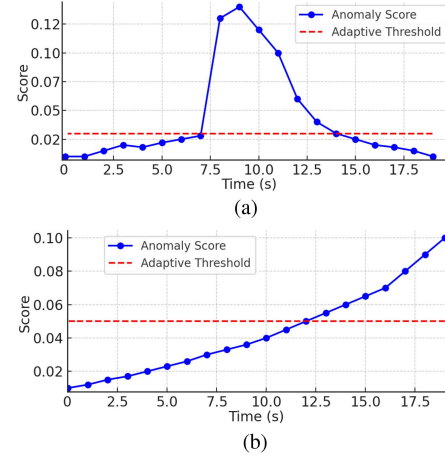


Fig. 7. Anomaly score $\tilde{a}_t$ and adaptive threshold τ_t during a sudden fault and a slow drift attack on FCC. (a) Sudden fault attack. Slow drift attack.

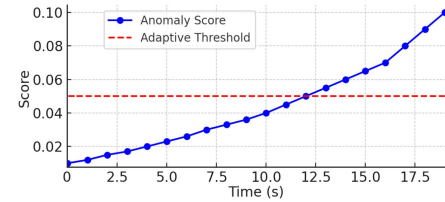


Fig. 8. Ablation study of trust threshold $r_{\min}$ on FCC. Lower values improve recall but reduce precision; $r_{\min} = 0.8$ provides the best trade-off.

affected period is then labeled anomalous, and updates resume only after correction, preserving model integrity.

On the BATADAL dataset, the proposed model outperformed the compared model and achieved a FAR = 2.2%, a detection accuracy of 0.89, and an F1-score of 0.90 in identifying the seven attack events. The recall was about 0.82, the only missed detections were for one very subtle slow drift attack, which our model learned as a new normal by the time it ended. The IDS performance against unseen attacks and adaptation to operational variations, without requiring labeled data, underscores its practical value for industrial deployment.

B. Ablation: Impact of Trust Threshold $r_{\min}$

The trust threshold $r_{\min}$ is a critical parameter. We tested values from 0.5 to 0.95 on the FCC dataset to see the effect.

As shown in Fig. 8, with $r_{\min} = 0.5$, the model was almost always updating (since half the time anomaly score is below

threshold by at least 50%). This led to a slight improvement in recall (it managed to adapt even more quickly to some distribution changes), but precision plummeted (false positives rose) because the model occasionally trained on points that were actually slight anomalies, thereby desensitizing itself momentarily. On the other hand, $r_{\min} = 0.95$ made the model very cautious; it rarely updated after the initial pretrain. This kept precision high but recall suffered as the model became stale under drift. We found $r_{\min} = 0.8$ to 0.85 a sweet spot in our case, giving a good balance and delivering the best F1-score. This aligns with the intuition that the model should be quite confident (80%–85%) in a point being normal to learn from it. In practice, $r_{\min}$ could be dynamically adjusted as well: for example, start conservative and if the false positive rate is very low, one might lower $r_{\min}$ to speed adaptation, until a false alarm happens then raise it. This is an interesting direction for future self-tuning of the trust threshold.

C. Comparison With Related Approaches

We compare our approach with several key data-driven and model-based IDS.

Comparison with Kitsune: Kitsune [12] is a widely cited online anomaly detector that relies on an ensemble of AEs, with each submodel trained on subsets of features. While effective on general network traffic, Kitsune requires careful tuning of ensemble size, thresholding parameters, and structural configuration of each AE. While our model employs a single latent-space predictive structure with a trust-based self-supervision mechanism, requiring fewer hyperparameters and lower memory overhead. In our case, training and inference remain fast, and retraining is triggered selectively based on confidence, which is better suited for resource-constrained industrial systems.

Trust-based learning: Gelenbe and Nakip [41] introduced statistical confidence intervals to assess trust in predictions and reduce the risk of concept drift corruption. Their work confirms the utility of trust estimation. Our method adapts this idea to time-series industrial settings by directly deriving trust from anomaly-to-threshold ratios, resulting in a lightweight but effective gating mechanism. Unlike their application to IoT traffic data, we validate our design on high-frequency ICS datasets.

Concept drift detectors: Techniques, such as ADWIN [42] and DDM [43], monitor score distributions to detect abrupt distribution changes. While theoretically sound, they often require parameter tuning (e.g., confidence windows) and DDM typically relies on labeled data for retraining triggers. Our trust mechanism achieves similar adaptive behavior using smoothed anomaly scores, without needing labels or predefined drift data. Furthermore, we avoid false retraining during small drifts.

Physics-based IDS: Model-based methods grounded in physical laws can achieve low FARs when accurate process models are available but are limited by predefined attack classes and domain. They require explicit modeling of each process and expert input for calibration. Although we do not use physical constraints, our data-driven model achieves near-equivalent precision on BATADAL and performs well on FCC, showing that careful statistical behavior modeling can be equally reliable while being generalizable.

Deep learning IDS (LSTM, graph neural network (GNN), and Transformers): Recent deep learning IDS architectures, such as recurrent neural network (RNN)–LSTM [44], GNN-based ICS IDS [45], and Transformer-based IDS [46], offer

TABLE IV
QUALITATIVE COMPARISON OF RELATED IDS APPROACHES

Method	Online	Label-Free	Adaptive	Generalizable
Kitsune [12]	✓	✓	✗	✓
Gelenbe and Nakip [41]	✓	✓	✓	✗
ADWIN [42]	✓	✓	✓	✓
DDM [43]	✓	✗	✓	✓
Physics-Based Models	✓	N/A	✗	✗
RNN–LSTM IDS [44]	✗	✗	✗	✓
GNN-Based ICS IDS [45]	✗	✗	✗	✓
Transformer IDS [46]	✗	✗	✗	✓
Ours	✓	✓	✓	✓

strong representational capacity but typically rely on supervised labels and offline retraining. They also lack online or trust-regulated adaptation, limiting their suitability for real-time industrial streaming settings. These approaches are included in Table IV for completeness, however, our numerical evaluations focus on IDS models that satisfy the same online, label-free operational constraints as our proposed framework. Tables II and III provide the corresponding quantitative comparisons.

D. Deployment Considerations for Industrial CPS

Deploying an ML-based IDS in an industrial environment raises additional practical considerations beyond raw detection performance. We address some of these aspects regarding our proposed framework.

Computational efficiency and edge deployment: Our model was intentionally kept lightweight (on the order of 10^4 parameters) to enable deployment on edge devices or embedded controllers commonly found in ICS. The average inference time per sample was 3.3 ms on a modern CPU without GPU acceleration, which is well within the cycle times of most industrial control loops. This enables real-time operation alongside control tasks. The occasional online update involves processing a batch of at most B samples. With $B = 256$, our update took 50–100 ms. In deployment, this update may be executed during a maintenance window, or B can be reduced for more frequent but faster updates. Overall, the approach is feasible on edge compute units; if an ICS already includes an SCADA server aggregating data, the IDS can be hosted there as well.

Integration with SCADA/human–machine interface (HMI): The IDS can be integrated into the SCADA layer, with alerts forwarded to the HMI. Since the model updates online and adjusts its parameters over time, transparency tools are essential. Displaying trust values, anomaly scores, and threshold evolution enables engineers to audit and interpret IDS behavior. Visualization overlays, such as Fig. 6, provide detailed insights into what the IDS is “seeing.” Operators retain control over adaptation via tunable parameters (e.g., $r_{\min}$) or by disabling updates if required.

Latency and response: For fast-acting attacks, detection latency is critical. Our method introduces a delay of L_t steps because it predicts L_t steps ahead. For our configuration ($L_t = 2$), this corresponds to a 2-s delay in FCC and a 2-h delay in BATADAL (due to hourly sampling), which is appropriate for water networks. If needed, then L_t can be set to 1 for near-immediate detection. In practice, $L_t = 2$ offered slightly smoother behavior. A hybrid option is to issue an early “warning” based on the first point in the target window and confirm after the complete window—reducing latency while preserving robustness. Smoothing and threshold computation introduce

only a one-sample delay. In high-speed domains (e.g., power grids), increasing α accelerates responsiveness.

Fail-safe behavior: If the IDS faces an attack that persistently evades the threshold, then the trust mechanism naturally halts updating (as r_t stays low). This produces a stable fail-safe: the detector stops adapting but continues monitoring without incorporating malicious data. Operators can monitor “trust inactivity” as a meta-signal; prolonged periods of low trust may trigger an external alert, such as “IDS confidence degraded, manual inspection recommended.”

Security of the IDS itself: An attacker might attempt a slow poisoning attack by feeding inputs that gradually retrain the model. The trust mechanism directly mitigates this risk: for a sample to influence training, its prediction error must remain significantly below the adaptive threshold. If the error is low, then the sample is, by definition, consistent with nominal behavior. The main requirement is that initial training data be normal; otherwise, the baseline could be compromised.

The proposed IDS framework is practical for industrial deployment with appropriate tuning. It operates in real time on modest hardware, integrates with standard monitoring tools, and includes mechanisms to control and supervise its learning behavior.

Planned hardware-in-the-loop (HIL) and real ICS validation: Beyond the FCC and BATADAL datasets, our evaluation includes three additional industrial datasets (CICIoT2023, WDT, and BATADAL-2), demonstrating that the IDS generalizes across diverse domains and sampling rates. However, all five datasets are simulation-based or process-level recordings and do not fully capture actuator nonlinearities, PLC/SCADA timing jitter, network congestion, or physical disturbances present in real facilities. To address this limitation, we have planned HIL validation on a PLC-driven process module in our laboratory, where the IDS will run in real time against a physical plant model. We also plan tests on a small-scale ICS testbed with Modbus/transmission control protocol (TCP) communication to evaluate latency, robustness under process noise, and engineering feasibility.

VII. CONCLUSION

This work introduced a self-supervised, context-target predictive intrusion detection framework for industrial CPS, enabling online anomaly detection without labeled data. By integrating a trust-aware adaptation mechanism, the model safeguards against learning from uncertain or malicious events while continuously adapting to process changes. Evaluated on FCC and BATADAL datasets, it achieved high detection performance with minimal delay and false alarms, outperforming conventional models. Its lightweight design supports real-time edge deployment, and the trust-based learning strategy adds robustness and interpretability. These results demonstrate the potential of the proposed model for securing industrial systems, with future directions, including multistep forecasting, real-world validation, and adaptive tuning mechanisms. In addition, we plan to conduct HIL and real ICS testbed experiments to further validate real-time adaptability and engineering feasibility beyond the current multidataset evaluation.

REFERENCES

- [1] T. Miller, A. Staves, S. Maesschalck, M. Sturdee, and B. Green, “Looking back to look forward: Lessons learnt from cyber-attacks on industrial control systems,” *Int. J. Crit. Infrastructure Protection*, vol. 35, Art. no. 100464, 2021.
- [2] J. P. Farwell and R. Rohozinski, “Stuxnet and the future of cyber war,” *Survival*, vol. 53, no. 1, pp. 23–40, 2011.
- [3] D. Bhamare, M. Zolanvari, A. Erbad, R. Jain, K. Khan, and N. Meskin, “Cybersecurity for industrial control systems: A survey,” *Comput. Secur.*, vol. 89, 2020, Art. no. 101677.
- [4] ICS Advisory Project and Industrial Data Works, “ICS[AP] and industrial data works analysis for 2023: ICS vulnerabilities” ICS Advisory Project and Industrial Data Works, Tech. Rep., 2024, Accessed: 2025-04-28. [Online]. Available: <https://tinyurl.com/37c8e29u>
- [5] S. Goel, “A systematic literature review on past attack analysis on industrial control systems,” *Trans. Emerg. Telecommun. Technol.*, vol. 35, no. 6, 2024, Art. no. e5004.
- [6] Y. Otoum and A. Nayak, “AS-IDS: Anomaly and signature based IDS for the Internet of Things,” *J. Netw. Syst. Manage.*, vol. 29, no. 3, 2021, Art. no. 23.
- [7] M. Asiri, N. Saxena, R. Gjomemo, and P. Burnap, “Understanding indicators of compromise against cyber-attacks in industrial control systems: A security perspective,” *ACM Trans. Cyber- Phys. Syst.*, vol. 7, no. 2, pp. 1–33, 2023.
- [8] F. Bayram, B. S. Ahmed, and A. Kessler, “From concept drift to model degradation: An overview on performance-aware drift detectors,” *Knowl.-Based Syst.*, vol. 245, Art. no. 108632, 2022.
- [9] V. Maurya, N. Rani, and S. K. Shukla, “RemOD: Operational drift-adaptive intrusion detection,” in *Security, Privacy, and Applied Cryptography Engineering*, ser. Lecture Notes in Computer Science, vol. 13783. Berlin, Germany: Springer, 2022, pp. 314–333.
- [10] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, “Anomal-e: A self-supervised network intrusion detection system based on graph neural networks,” *Knowl.-Based Syst.*, vol. 258, 2022, Art. no. 110030.
- [11] A. Binbusayyis and T. Vaiyapuri, “Unsupervised deep learning approach for network intrusion detection combining convolutional autoencoder and one-class SVM,” *Appl. Intell.*, vol. 51, no. 10, pp. 7094–7108, 2021.
- [12] Y. Minsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb. 2018, doi: [10.14722/ndss.2018.23204](https://doi.org/10.14722/ndss.2018.23204).
- [13] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding,” in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [14] B. Israelsen, N. Ahmed, E. Frew, D. Lawrence, and B. Argrow, “Machine self-confidence in autonomous systems via meta-analysis of decision processes,” in *Proc. AHFE Int. Conf. Hum. Factors AI Syst. Eng.*, 2020, pp. 213–223.
- [15] E. C. Balta, M. Pease, J. Moyne, K. Barton, and D. M. Tilbury, “Digital twin-based cyber-attack detection framework for cyber-physical manufacturing systems,” *IEEE Trans. Automat. Sci. Eng.*, vol. 21, no. 2, pp. 1695–1712, Apr. 2024.
- [16] M. R. G. Raman and A. P. Mathur, “A hybrid physics-based data-driven framework for anomaly detection in industrial control systems,” *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 52, no. 9, pp. 6003–6014, Sep. 2022.
- [17] J. Giraldo et al., “A survey of physics-based attack detection in cyber-physical systems,” *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–36, 2018.
- [18] J. Suaboot et al., “A taxonomy of supervised learning for IDSs in SCADA environments,” *ACM Comput. Surv.*, vol. 53, no. 2, pp. 1–37, 2020.
- [19] O. A. Alimi, K. Ouahada, A. M. Abu-Mahfouz, S. Rimer, and K. O. A. Alimi, “A review of research works on supervised learning algorithms for SCADA intrusion detection and classification,” *Sustainability*, vol. 13, no. 17, 2021, Art. no. 9597.
- [20] S. S. A. Naqvi, Y. Li, and M. Uzair, “DDoS attack detection in smart grid network using reconstructive machine learning models,” *PeerJ Comput. Sci.*, vol. 10, 2024, Art. no. e1784.
- [21] S. Ali and Y. Li, “Learning multilevel auto-encoders for DDoS attack detection in smart grid network,” *IEEE Access*, vol. 7, pp. 108647–108659, 2019.
- [22] X. Du, C. Zhou, Y.-C. Tian, and K. Wang, “Anomaly detection based on data super-resolution in industrial cyber-physical systems with multi-rate sampling,” *IEEE Sensors J.*, vol. 24, no. 10, pp. 16478–16490, May 2024.
- [23] A. S. Musleh, G. Chen, Z. Y. Dong, C. Wang, and S. Chen, “Attack detection in automatic generation control systems using LSTM-based stacked autoencoders,” *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 153–165, 2022.
- [24] F. Turrin, A. Erba, N. O. Tippenhauer, and M. Conti, “A statistical analysis framework for ICS process datasets,” in *Proc. Joint Workshop CPSIoT Secur. Privacy*, 2020, pp. 25–30.

- [25] A. Erba and N. O. Tippenhauer, "Assessing model-free anomaly detection in industrial control systems against generic concealment attacks," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, 2022, pp. 412–426.
- [26] C. Hu, L. Zhu, and S. Lai, "Spatio-temporal-based context fusion for video anomaly detection," in *Proc. Int. Conf. Pattern Recognit., Mach. Vis. Intell. t Algorithms (PRMVIA)*, 2023, pp. 187–192.
- [27] J. Liu et al., "Context2Vector: Accelerating security event triage via context representation learning," *Inf. Softw. Technol.*, vol. 146, 2022, Art. no. 106856.
- [28] M. A. Shyaa, Z. Zainol, R. Abdullah, M. Anbar, L. Alzubaidi, and J. Santamaría, "Enhanced intrusion detection with data stream classification and concept drift guided by the incremental learning genetic programming combiner," *Sensors*, vol. 23, no. 7, 2023, Art. no. 3736.
- [29] Z. Yang, S. Al-Dahidi, P. Baraldi, E. Zio, and L. Montelatici, "A novel concept drift detection method for incremental learning in nonstationary environments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 1, pp. 309–320, 2019.
- [30] M. Nakip and E. Gelenbe, "Online self-supervised deep learning for intrusion detection systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5668–5683, 2024.
- [31] E. Gelenbe and M. Nakip, "IoT network cybersecurity assessment with the associated random neural network," *IEEE Access*, vol. 11, pp. 85501–85512, 2023.
- [32] R. Sun, S. Zhang, C. Yin, J. Wang, and S. Min, "Strategies for data stream mining method applied in anomaly detection," *Cluster Comput.*, vol. 22, pp. 399–408, 2019.
- [33] S. A. Varghese, A. Dehlaghi-Ghadim, A. Balador, Z. Alimadadi, and P. Papadimitratos, "Digital twin-based intrusion detection for industrial control systems," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops Other Affiliated Events (PerCom Workshops)*, 2022, pp. 611–617.
- [34] O. Santander, V. Kuppura, C. A. Harrison, and M. Baldea, "An open source fluid catalytic cracker–fractionator model to support the development and benchmarking of process control, machine learning and operation strategies," *Comput. Chem. Eng.*, vol. 164, 2022, Art. no. 107900. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0098135422002381>
- [35] S. S. A. Naqvi, C. Zhou, P. Xu, Y. Li, J. Jiashu, and M. Uzair, "Adversarial feature generation for ML-based intrusion detection in the petrochemical industry," *J. Inf. Secur. Appl.*, vol. 94, 2025, Art. no. 104215.
- [36] R. Taormina et al., "The battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks," *J. Water Resour. Plan. Manage.*, vol. 144, no. 8, Aug. 2018, Art. no. 0 4018048.
- [37] B. Brentan, P. Rezende, D. Barros, G. Meirelles, E. Luvizotto Jr, and J. Izquierdo, "Cyber-attack detection in water distribution systems based on blind sources separation technique," *Water*, vol. 13, no. 6, 2021, Art. no. 795.
- [38] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, 2023, Art. no. 5941.
- [39] L. Faramondi, F. Flammini, S. Guarino, and R. Setola, "A hardware-in-the-loop water distribution testbed dataset for cyber-physical security testing," *IEEE Access*, vol. 9, pp. 122385–122396, 2021.
- [40] A. Erba, A. F. Murillo, R. Taormina, S. Galelli, and N. O. Tippenhauer, "On practical realization of evasion attacks for industrial control systems," in *Proc. Workshop Re-Des. Ind. Control Syst. with Secur.*, 2023, pp. 9–25.
- [41] E. Gelenbe and M. Nakip, "Traffic based sequential learning during botnet attacks to identify compromised IoT devices," *IEEE Access*, vol. 10, pp. 126536–126549, 2022.
- [42] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, 2007, pp. 443–448.
- [43] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. Adv. Artif. Intell.—SBIA 2004: 17th Braz. Symp. Artif. Intell.*, Sao Luis, Brazil, Sep. 29–Oct. 1, 2004, pp. 286–295.
- [44] A. Alexandrov, "LSTM-RNN method for anomaly-based intrusion detection systems," in *Proc. BISEC*, 2024, pp. 17–33.
- [45] L. Shuaiyi, K. Wang, Y. Wei, H. Liu, Q. Fan, and B. Wang, "GNN-based advanced feature integration for ICS anomaly detection," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 6, pp. 106–1, 2023.
- [46] U. C. Akuthota and L. Bhargava, "Transformer based intrusion detection for IoT networks," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 6062–6067, Mar. 2025.



Sardar Shan Ali Naqvi received the B.S. and M.S. degrees in electrical (computer) engineering from COMSATS University Islamabad, Wah Campus, Islamabad, Pakistan, in 2016, and the Ph.D. degree in information security from North China Electric Power University, Beijing, China, in 2024.

He is currently a Postdoctoral Research Fellow with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China. His research interests include intrusion detection, network attacks, security control of industrial control systems, and artificial intelligence.



Chunjie Zhou received the M.S. and Ph.D. degrees in control theory and control engineering from the Huazhong University of Science and Technology, Wuhan, China, in 1991 and 2001, respectively.

He is currently a Professor with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology. His research interests include safety and security control of industrial control systems, theory and application of networked control systems, and

artificial intelligence.



Peihang Xu (Student Member, IEEE) received the B.Eng. degree in mechanical engineering from the Nanchang Institute of Technology, Nanchang, China, in 2018, and the M.Eng. degree in mechanical engineering from the School of Mechanical Engineering, Guangxi University, Nanning, China in 2022. He is currently working toward the Ph.D degree in control science and engineering with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China.

His research interests include the security requirements elicitation, system-level modeling, vulnerability identification, fuzzing, and failure analysis of cyber security.



Yahui Li received the M.Eng. degree in automation from Hangzhou Dianzi University, Hangzhou, China, in 2019. He is currently working toward the Ph.D. degree in artificial intelligence with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China.

His research interests include industrial cyber-physical system, artificial intelligence, and knowledge graph.



Muhammad Uzair received the M.S. degree in electronics and computer engineering from Hanyang University, Seoul, South Korea, in 2007, and the Ph.D. degree in computer engineering from the University of Western Australia, Crawley, WA, Australia, in 2016.

He was an Assistant Professor in electrical engineering with COMSATS University Islamabad, Islamabad, Pakistan, from 2016 to 2018, and later as a Research Associate with the University of South Australia, Adelaide, SA, Australia, from 2018 to 2021. He was also a Software Engineer with Topcon Precision Systems, Adelaide. He is currently a Lecturer with the School of Computer and Mathematical Sciences, University of Adelaide, Adelaide. His research interests include machine learning, data analytic, computer vision, and biologically inspired signal processing.